Technical University of Denmark

DTU Compute
Department of Applied Mathematics and Computer Science

Advanced SQL Programming

Sections 4.3, 5.2, 5.3 in the textbook

©Anne Haxthausen and Flemming Schmidt

These slides have been prepared by Anne Haxthausen, partly reusing/modifying slides by Flemming Schmidt. Some examples come from Silberschatz, Korth, Sudarshan, 2010.

Contents

- Procedural Statements
- Programming Objects
  - Functions
  - Procedures
  - Triggers
  - Transactions
  - Events
- Errors and Warnings
  - Error Handling
- SQL Access from a Programming Language
  - ODBC, JDBC and Embedded SQL
- Demo Exercises and Exercises
- Appendix: University Database

SQL Statements

- Declarative statements, e.g.:
  - DDL statements like CREATE and DROP
  - DML statements like INSERT, DELETE and SELECT

- Procedural/imperative statements:
  - Variable assignments
  - If-then-else, case
  - Various kinds of loops
  - Blocks (BEGIN … END)
  - Function and procedure calls
  - Return statement (inside a function body)
  - …

  Note: Most DBMS implement non standard syntax for procedural statements, but the concepts are the same! These slides use the syntax implemented by MySQL/MariaDB (while the text book uses the standard syntax).

Procedural Statements

- BEGIN-END blocks

  [*label_name*:] **BEGIN**

  local-variable-declarations #scope is within the begin-end block

  statements

  END [*label_name*] ;

  - *local-variable-declarations* are of the form:

    DECLARE *var_name type* [**DEFAULT** *value*] ;

- Variable assignments

  SET *var_name = expr;*

  - where *var_name* can be the name of
    - a *local variable* declared in an enclosed BEGIN-END statement
    - a formal *parameter* of an enclosed procedure (or function)
    - a *user-defined variable* (name is of the form @id, no specific declaration, is session specific),
      e.g. **SET** @x = 1; **SET** @x = @x +1;

  SET [**GLOBAL**] *system_var_name = value;*

- Variable assignments in SELECT clauses

  SELECT …, *expr* **INTO** *var_name* , … ….

  Example: **SELECT** SUM(Salary) **INTO** @var **FROM** Instructor; **SELECT** @var;

@var
999000.00

Procedural Statements

- IF … THEN … ELSEIF … THEN … ELSE … END IF;

  IF *condition*$_1$ **THEN** *statements*$_1$
  [**ELSEIF** *condition*$_2$ **THEN** *statements*$_2$]
  …
  [**ELSE** *statements*$_n$]
  END IF;

Procedural Statements

- CASE using an expression or conditions

CASE *expression*
WHEN $value_1$ **THEN**
    $statements_1$ #to execute when expression equals value_1
...
WHEN $value_n$ **THEN**
    $statements_n$ #to execute when expression equals value_n
[**ELSE**
    statements #to execute when no values matched]
END CASE;

CASE
WHEN $condition_1$ **THEN**
    $statements_1$ #to execute when condition_1 is TRUE
...
WHEN $condition_n$ **THEN**
    $statements_n$ #to execute when condition_n is TRUE
[**ELSE**
    statements #to execute when all conditions were FALSE]
END CASE;

Procedural Statements

- LOOP:

  [*label_name*:] **LOOP**
  statements #can be terminated by a LEAVE or RETURN statement
  END LOOP [*label_name*];

- WHILE DO:

  [*label_name*:] **WHILE** *condition* **DO**
  statements
  END WHILE [*label_name*];

- REPEAT UNTIL:

  [*label_name*:] **REPEAT**
  statements
  UNTIL *condition*
  END REPEAT [*label_name*];

- LEAVE: **LEAVE** *label_name* # to exit a labelled loop

- ITERATE: **ITERATE** *label_name* # to start a labelled loop again

Programming Objects

- **A Function**
  - is a stored program for which parameters can be passed in, and then it will return a value:
    CREATE FUNCTION function_name (parameter$_1$ datatype$_1$,…,parameter$_n$ datatype$_n$)
    RETURNS return_datatype
    routine_body #Must include a 'RETURN value' statement

- **A Procedure**
  - is a stored program for which parameters can be passed in and out:
    CREATE PROCEDURE procedure_name ([**IN**|**OUT**|**INOUT**] parameter$_1$ datatype$_1$, …)
    routine_body

- **A Trigger**
  - is a stored program that is executed BEFORE/AFTER an INSERT, UPDATE or DELETE operation:
    CREATE TRIGGER trigger_name #Use naming standard like: Table_name_Before_Insert
    { **BEFORE** | **AFTER** } { **INSERT** | **UPDATE** | **DELETE** }
    ON table_name **FOR EACH ROW**
    actions

Programming Objects

- **A Transaction**
  - Is a group of SQL statements that executes as one entity. If one or more statements fails to execute successfully, then the successfully executed statements are undone, and the database is returned to the initial state before the transaction was started. In practice, all changes are stored in temporary storage, until they are either committed and stored permanently, or discarded (i.e. rolled back) and deleted from temporary storage.
  - Transactions can be executed concurrently by the DBMS without disturbing each other.
- **An Event**
  - Is a stored program that is executed at a given time or at given time intervals:
    CREATE EVENT event_name
    ON SCHEDULE schedule
    DO statement
- **SQL Access from a Programming Language**
  - Normally the application user interface and application logic are programmed using a common programming language like C, Java or Cobol.
  - Application Programs written in a programming language can interact with a database using an Application Program Interface (API) for example ODBC and JDBC.
  - The SQL standard also defines embedding's of SQL in a variety of programming languages such as C, Java, and Cobol, for example like inserting in the programming language code:
    EXEC SQL <embedded SQL statement > END_EXEC

Functions

- Create a function

  CREATE FUNCTION *function_name* (*parameter*$_1$ *datatype*$_1$, … , *parameter*$_n$ *datatype*$_n$)
  RETURNS *return_datatype*

  routine_body #must include a 'RETURN value' statement

  - A *routine_body* is a *statement* (can e.g. be a block).
  - Must not be recursive.

- Call a function

  function_name(e_1, …, e_n)

  - e_1, …, e_n are expressions matching the formal parameters.
  - Is syntactically a *value expression.*

- Drop a function

  DROP FUNCTION *function_name*;

- Note, if binary logging is enabled in your MySQL/MariaDB server (you can check this with the command **SHOW VARIABLES LIKE** 'log_bin'; which replies 'ON' if it is enabled) you may need to execute **SET GLOBAL** log_bin_trust_function_creators = 1; in order to be able use functions. Otherwise (when the reply is 'OFF'), it should not be needed.

Functions – Example

- Create a function calculating Age from Date of Birth

```
CREATE FUNCTION Age (vDate DATE) RETURNS INTEGER
RETURN TIMESTAMPDIFF(YEAR, vDate, CURDATE());
```

- Test the function

```
SELECT Age(20000720);
```

Age(20000720)
20

```
SELECT StudID, StudName, Birth, Age(Birth) FROM Student;
```

StudID	StudName	Birth	Age(Birth)
00128	Zhang	1992-04-18	23
12345	Shankar	1995-12-06	20
19991	Brandt	1993-05-24	22
23121	Chavez	1992-04-18	23
44553	Peltier	1995-10-18	20
45678	Levy	1995-08-01	20
54321	Williams	1995-02-28	20

Functions - Example

- Create a function and test it
  - Given a department name, the function returns the number of instructors

```
DELIMITER //
CREATE FUNCTION DeptInstCount (vDeptName VARCHAR(20)) RETURNS INT
BEGIN
  DECLARE vDeptInstCount INT;
  SELECT COUNT(*) INTO vDeptInstCount FROM Instructor
  WHERE DeptName = vDeptName;
  RETURN vDeptInstCount;
END//
DELIMITER ;
```

DeptName	Instructors
Biology	1
Comp. Sci.	3
Elec. Eng.	1
Finance	2
History	2
Music	1
Physics	2

The SQL DELIMITER is temporarily changed from ";" to "//" in order to allow ";" in the function body!

- **SELECT** DeptName, DeptInstCount(DeptName) **AS** Instructors **FROM** Department;
- Find department name and budget of all departments with two or more instructors.

```
SELECT DeptName, Budget FROM Department
WHERE DeptInstCount(DeptName) >= 2;
```

DeptName	Budget
Comp. Sci.	100000.00
Finance	120000.00
History	50000.00
Physics	70000.00

Procedures

- Create a procedure

  CREATE PROCEDURE *procedure_name*

  (**[IN|OUT|INOUT]** $parameter_1$ $datatype_1$, …, **[IN|OUT|INOUT]** $parameter_n$ $datatype_n$) *routine_body*

  - A *routine_body* is a *statement* (typically a block).
  - An **IN** parameter passes a value into a procedure.
  - An **OUT** parameter passes a value from the procedure back to the caller.
  - An **INOUT** parameter passes a value into the procedure and back to the caller.
  - Note that a procedure can have several output parameters!

- Call a procedure

  CALL *procedure_name*(e_1, …, e_n);

  - e_1, …, e_n are expressions matching the formal parameters wrt types.
  - When $parameter_i$ *is* **OUT** and **INOUT** parameters, e_i must be a variable name.
  - Is syntactically a *statement.*
  - **CALL** *p*(); is equivalent to **CALL** *p*;

- Drop a procedure

  DROP PROCEDURE *procedure_name*;

Procedures – Examples Without Parameters

- Create a simple procedure and test it

```
CREATE PROCEDURE ShowVersion() SELECT VERSION() AS 'My DBMS Version';
CALL ShowVersion;
```

My DBMS Version
10.3.12-MariaDB

- Create a more elaborate procedure (providing a lucky number, which is 4 or 7)

```
DELIMITER $$
CREATE PROCEDURE LuckyNumber()
BEGIN
  DECLARE vNumber INTEGER DEFAULT 0;
  IF DAY(CURRENT_DATE())%2 = 0 THEN SET vNumber = 4; ELSE SET vNumber = 7; END IF;
  SELECT vNumber AS 'Todays Lucky Number';
END $$
DELIMITER ;
```

```
CALL LuckyNumber;
```

Todays Lucky Number
4

Procedures – Example with IN Parameters and Side Effects

- Create a procedure
  - to add a new instructor with the minimum wage 29000 as salary:

```
DELIMITER //
CREATE PROCEDURE AddInstructor
   (IN vInstID VARCHAR(5), IN vInstName VARCHAR(20), IN vDeptName VARCHAR(20))
BEGIN
  INSERT Instructor(InstID, InstName, DeptName, Salary)
       VALUES (vInstID, vInstName, vDeptName, 29000);   #As a side effect, a table is changed
END //
DELIMITER ;
```

  - This procedure has: 3 inputs, 0 outputs.
  - As a side effect it changes the Instructor table.

- Use a CALL statement to execute the procedure

```
CALL AddInstructor ('99999', 'Schmidt', 'Comp. Sci.');
SELECT * FROM Instructor;
```

InstID	InstName	DeptName	Salary
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00
99999	Schmidt	Comp. Sci.	29000.00
NULL	NULL	NULL	NULL

Procedures – Example with IN and OUT parameters

- Create a procedure
  - to calculate the maximum capacity for a building:

```
DELIMITER //
CREATE PROCEDURE BuildingCapacity
(IN vBuilding VARCHAR(20), OUT vMaxCapacity INT)
BEGIN
  SELECT SUM(Capacity) INTO vMaxCapacity FROM Classroom
  WHERE Building = vBuilding;
END //
DELIMITER ;
```

Classroom

Building	Room	Capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

  - This procedure has: 1 input variable, 1 output variable, no side effects on tables.
- Call the procedure

```
CALL BuildingCapacity ('Watson', @MaxCap);
```

After this call the output variable can be read:

  - **SELECT** @MaxCap;

@MaxCap
80

  - **SELECT** * **FROM** Classroom **WHERE** Capacity > @MaxCap;

Building	Room	Capacity
Packard	101	500

Triggers

- A Trigger
  - Is a set of SQL statements that are *executed automatically* by the DBMS system as *a side effect of a table modification* (i.e. an SQL INSERT, UPDATE or DELETE).
  - Triggers can be used to *preprocess* and *post process* table changes.
    - Example: In order to *validate values* of a new row, a trigger can be executed BEFORE a row is inserted in a table, and it might reject insertion.
    - Example: In order to update *log tables*, a trigger can be executed AFTER a row has been inserted in a table.
    - Example: In order to *enforce integrity constraints* triggers may process tables changes.

Triggers

- Create a trigger:

  CREATE TRIGGER *trigger_name* #Use naming standard like: table_name_Before_Insert
  { **BEFORE** | **AFTER** } { **INSERT** | **UPDATE** | **DELETE** } **ON** *table_name*
  FOR EACH ROW *actions* # trigger actions

  - *actions* is a statement.
  - **OLD**.*A* and **NEW**.*A* can be used in *actions* to refer to attribute *A* before an (UPDATE or DELETE) event and after an (UPDATE or INSERT) event, respectively.
  - In a BEFORE trigger, you can make assignments **SET NEW**.*A* = … . This means you can use a trigger to modify the values to be inserted into a new row or used to update a row.
- The trigger definition specifies
  - the actions to be taken for inserted/updated/deleted rows when the trigger executes
    - IMPORTANT: Note that the "FOR EACH ROW" refers only to each row inserted/updated/deleted by the original query, not every row of the table! So if you only inserted one row, your trigger will run once in the context of that row.
  - the event for which a trigger is to be executed: an INSERT, UPDATE or DELETE
  - the time at which the trigger should be executed: BEFORE or AFTER the event
- Drop a trigger:
  - **DROP TRIGGER** *table_name.trigger_name;* # *table_name.* only needed if *trigger_name* is defined on several tables.
- Show triggers in a database named *db:*
  - **SHOW TRIGGERS IN** *db;*

Triggers – Example With Preprocessing

- Create a trigger that executes **before** a row is Inserted in Instructor
  - The trigger statement below is executed before a row is inserted in table Instructor. Statements with INSERT involving the Instructor table will trigger execution of the trigger statement (see next slide).

```
DELIMITER //
CREATE TRIGGER Instructor_Before_Insert
BEFORE INSERT ON Instructor FOR EACH ROW
BEGIN
  IF NEW.Salary < 29000 THEN SET NEW.Salary = 29000;
  ELSEIF NEW.Salary > 150000 THEN SET NEW.Salary = 150000;
  END IF;
END//
DELIMITER ;
```

  - A trigger can also be used to make a log of inserts since last backup. **After** a row is inserted in the table Instructor the same row is inserted in the table InstLog. Logs are then cleared after backup. See demo exercise #2.
  - A trigger that raises a signal (exception) for undesired insertions will be shown under the Error Handling part of these slides.

Trigger Test of Instructor_Before_Insert

INSERT Instructor **VALUES** (99997, 'Anne', 'Comp. Sci.', 500000.00); #too high a salary

INSERT Instructor **VALUES** (99998, ' Helen', 'Comp. Sci.', 100000.00);

INSERT Instructor **VALUES** (99999, ' Peter', 'Comp. Sci.', 20000.00); #too low a salary

SELECT InstName, Salary **FROM** Instructor **WHERE** DeptName = 'Comp. Sci.';

InstName	Salary
Srinivasan	65000.00
Katz	75000.00
Brandt	92000.00
Anne	150000.00
Helen	100000.00
Peter	29000.00

Triggers – Example with Postprocessing

Given Takes(StudID, CourseID, …, Grade), Student(StudID, …, TotCredits), Course(CourseID, …, Credits)

- Create a trigger that updates Student **after** a row has been updated in Takes: it should bring the total credits (TotCredits) of a student up-to-date when the student passes a course.

```
DELIMITER $$
CREATE TRIGGER Takes_After_Update
AFTER UPDATE ON Takes FOR EACH ROW
IF NEW.Grade <> 'F' AND NEW.Grade IS NOT NULL  # course is passed after
   AND
   (OLD.Grade = 'F' or OLD.Grade IS NULL)  # course was not passed before
THEN  UPDATE Student
        SET TotCredits = TotCredits +
           (SELECT Credits FROM Course WHERE Course.CourseID = NEW.CourseID)
        WHERE Student.StudID = NEW.StudID;
END IF $$
DELIMITER ;
```

Test Trigger Takes_After_Update

SELECT * FROM Takes **WHERE** StudID = 98988;

StudID	CourseID	SectionID	Semester	StudyYear	Grade
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	NULL

SELECT StudID, TotCredits **FROM** Student **WHERE** StudID = 98988;

StudID	TotCredits
98988	120

UPDATE Takes **SET** Grade = 'A' **WHERE** StudID = 98988 **AND** CourseID = 'BIO-301';

SELECT * FROM Takes **WHERE** StudID = 98988;

StudID	CourseID	SectionID	Semester	StudyYear	Grade
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	A

SELECT StudID, TotCredits **FROM** Student **WHERE** StudID = 98988;

StudID	TotCredits
98988	124

When Not To Use Triggers!

- Triggers were used earlier for tasks such as:
  - *Maintaining summary data* tables (e.g., total salary of each department).
  - *Replicating databases* by recording changes to special relations (called change or delta relations) and having a separate process that applies the changes over to a replica.

- There are better ways of doing these now:
  - Databases today provide built-in View facilities to maintain summary data.
  - Databases provide built-in support for replication.

Transactions

- SQL Transaction
  - Is a group of SQL statements (like SELECT, INSERT, UPDATE and DELETE), that *executes as one entity*.
  - If just one statement *fails* to execute successfully, then the successfully executed statements are *rolled back*, and the database is returned to the initial state before the transaction was started.
  - Transactions *can be executed concurrently* without disturbing each other.

- Nature or transactions
  - All transactions have a beginning and an end.
  - Transaction results are either saved (*committed*) or aborted (*rollback*).
  - If a transaction fails, then no part of the transaction is saved.

Transactions – Motivating Example

- Moving amounts from one bank account to another
  - Amounts need to be moved without being lost or moved twice!

SELECT * FROM Accounts;

ID	Owner	Amount
1001	James Morrison	35000.00
1002	Frank Summers	15000.00
1003	Hank Williamson	40000.00

  - Transfer 5.000 from Frank (ID 1002) to Hank (ID 1003)

```
SET @Transfer = 5000;
SET @OldAmountID1 = (SELECT Amount FROM Accounts WHERE ID = 1002);
UPDATE Accounts SET Amount = (@OldAmountID1 - @Transfer) WHERE ID = 1002;
SET @OldAmountID2 = (SELECT Amount FROM Accounts WHERE ID = 1003);
UPDATE Accounts SET Amount = (@OldAmountID2 + @Transfer) WHERE ID = 1003;
```

SELECT * FROM Accounts;

ID	Owner	Amount
1001	James Morrison	35000.00
1002	Frank Summers	10000.00
1003	Hank Williamson	45000.00

However, the database would loose 5.000 if the second update failed. If ID 1003 did not exist, or if the system crashed, of if another user or program interfered between the updates!

Controlling Transactions

- START TRANSACTION;
  - Will ensure that all row inserts, updates and deletes are stored in a Temporary Buffer and NOT written to disk and permanently stored.
- COMMIT;
  - Will ensure that the changes stored in the Temporary Buffer are written to disk and permanently stored.
- ROLLBACK;
  - Will ensure that all changes made since the START TRANSACTION will be deleted in the Temporary Buffer and NOT written to disk and permanently stored.

Procedure with a Transaction

- Create a Procedure that includes a Transaction

```
DELIMITER //
CREATE PROCEDURE Transfer (
        IN vTransfer DECIMAL(9,2), vID1 INT, vID2 INT,  OUT vStatus VARCHAR(45))
BEGIN
  DECLARE OldAmountID1, NewAmountID1, OldAmountID2, NewAmountID2 INT DEFAULT 0;
  START TRANSACTION;
  SET SQL_SAFE_UPDATES = 0;
  SET OldAmountID1 = (SELECT Amount FROM Accounts WHERE ID = vID1);
  SET NewAmountID1 = OldAmountID1 - vTransfer;
  UPDATE Accounts SET Amount = NewAmountID1 WHERE ID = vID1;
  SET OldAmountID2 = (SELECT Amount FROM Accounts WHERE ID = vID2);
  SET NewAmountID2 = OldAmountID2 + vTransfer;
  UPDATE Accounts SET Amount = NewAmountID2 WHERE ID = vID2;
  IF (OldAmountID1 + OldAmountID2) = (NewAmountID1 + NewAmountID2)
    THEN SET vStatus = 'Transaction Transfer committed!';  COMMIT;
    ELSE SET vStatus = 'Transaction Transfer rollback'; ROLLBACK;
  END IF;
END //
DELIMITER ;
```

Testing Procedure with Transaction

SELECT * FROM Accounts;

ID	Owner	Amount
1001	James Morrison	35000.00
1002	Frank Summers	15000.00
1003	Hank Williamson	40000.00

CALL Transfer (5000, 1002, 1003, @TStatus);
SELECT @TStatus;

@TStatus
Transaction Transfer committed!

SELECT * **FROM** Accounts;

ID	Owner	Amount
1001	James Morrison	35000.00
1002	Frank Summers	10000.00
1003	Hank Williamson	45000.00

CALL Transfer (5000, 1002, 1015, @TStatus);
SELECT @TStatus;

@TStatus
Transaction Transfer rollback!

CALL Transfer (5000, 1012, 1015, @TStatus);
SELECT @TStatus;

@TStatus
Transaction Transfer rollback!

SELECT * FROM Accounts;

ID	Owner	Amount
1001	James Morrison	35000.00
1002	Frank Summers	10000.00
1003	Hank Williamson	45000.00

Events

- CREATE an event

  CREATE EVENT *event_name*
  ON SCHEDULE *schedule*
  DO *statement*

  - *schedule* can e.g. be of the form

    EVERY *n timeunit*

    [**STARTS** *timestamp*]

    [**ENDS** *timestamp*]
  - *timeunit* can be: YEAR | QUARTER | MONTH | DAY | HOUR | MINUTE | WEEK | SECOND | YEAR_MONTH | DAY_HOUR | DAY_MINUTE | DAY_SECOND | HOUR_MINUTE | HOUR_SECOND | MINUTE_SECOND
  - **SET GLOBAL** event_scheduler = 1; # in order for MariaDB/MySQL to execute events
- Events can e.g. be used to schedule database period backups at regular time intervals.

Events - Example

- Setting up a database operation that runs periodically

```
SHOW VARIABLES LIKE 'event_scheduler';          # Initially it is set OFF
```

Variable_name	Value
event_scheduler	OFF

```
SET GLOBAL event_scheduler = 1;          # To set it ON. MySQL will look for/execute events

CREATE TABLE MarkLog(
        TS              TIMESTAMP,
        Message     VARCHAR(100));

CREATE EVENT MarkInsert
ON SCHEDULE EVERY 2 MINUTE
DO INSERT MarkLog VALUES (CURRENT_TIMESTAMP, "-- It's time again--");

SELECT * FROM MarkLog;
```

TS	Message
2015-07-07 21:14:04	-- It's time again--
2015-07-07 21:16:04	-- It's time again--
2015-07-07 21:18:04	-- It's time again--

Error Handling

- MariaDB raises signals and show Warnings & Error Messages for predefined conditions (cases).
- Examples using the MariaDB Command Line Client:

```
Enter password: ****
MariaDB[none]> select * from instructor;
ERROR 1046 (3D000): No database selected

MariaDB[none]> use University;
Database changed

MariaDB[University]> select * from instructor table;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use
near 'table' at line 1

MariaDB[University]> select * from instructors;
ERROR 1146 (42S02): Table 'university.instructors' doesn't exist

MariaDB[University]> select * from instructor;
+--------+------------+------------+----------+
| InstID | InstName   | DeptName   | Salary   |
+--------+------------+------------+----------+
| 10101  | Srinivasan | Comp. Sci. | 65000.00 |

| 98345  | Kim        | Elec. Eng. | 80000.00 |
…
12 rows in set (0.02 sec)
```

Error Messages

- Example: ERROR 1146 (42S02): Table 'university.instructors' doesn't exist
  - 1146 is a MySQL/MariaDB error code. The set of MariaDB error codes is a superset of the set of MySQL error codes.
  - Table 'university.instructors' doesn't exist is the associated error text.
  - The error codes and texts are not portable to other DBMS!
  - 42S02 is the SQLSTATE used by ANSI SQL and ODBC. It gives error information:
    - Not all MariaDB error codes are mapped to SQLSTATE error codes.
    - HY000 is used for a general error for unmapped error numbers.
    - 00000 indicates successful completion of an SQL statement.
    - …
- For info about the codes, see in the MariaDB documentation:
  - https://mariadb.com/kb/en/library/mariadb-error-codes/

User-defined Error Signalling

- It is possible to raise signals for user-defined conditions indicating errors and warnings using the SIGNAL statement. Typical pattern for this:

  IF *condition* **THEN SIGNAL SQLSTATE** *state*

  SET MYSQL_ERRNO = …, **MESSAGE_TEXT** = …;

- Example from the body of the Course_Before_Insert trigger on next page:

```
IF NOT (NEW.Credits BETWEEN 0 AND 5)
 THEN SIGNAL SQLSTATE 'HY000'
   SET MYSQL_ERRNO = 1525, MESSAGE_TEXT = 'Invalid Credits; Range is [0,5]!';
 END IF;
```

  - SQLSTATE HY000 means that a general error has occurred
  - MYSQL_ERRNO 1525 means wrong value occurred

  - *When the signal is raised*, the trigger program is aborted and the related INSERT is not executed!

Example: Trigger With Error Handling and Preprocessing

- Trigger with rejection and preprocessing

CourseID	Title	DeptName	Credits
BIO-101	Intro. to Biology	Biology	4

Rejection if CourseID format is invalid or Credits not in [0,5].
Preprocessing of Title by removing leading blanks.

CourseID
BIO-101
BIO-301
BIO-399
CS-101
CS-190
CS-315
CS-319
CS-347
EE-181
FIN-201
HIS-351
MU-199
PHY-101

```
DELIMITER $$
CREATE TRIGGER Course_Before_Insert
BEFORE INSERT ON Course FOR EACH ROW
BEGIN
  IF NOT (
    ( (LENGTH(NEW.CourseID)=6 AND LEFT(NEW.CourseID,2) IN ('CS','EE','MU'))
      OR
      (LENGTH(NEW.CourseID)=7 AND LEFT(NEW.CourseID,3) IN ('BIO','FIN','HIS','PHY'))
    )
    AND LOCATE('-',NEW.CourseID) <> 0
    AND RIGHT(NEW.CourseID,3) BETWEEN 100 AND 999 )
  THEN SIGNAL SQLSTATE 'HY000'
      SET MYSQL_ERRNO = 1525, MESSAGE_TEXT = 'Invalid CourseID; Format is LL(L)-DDD';
  END IF;
  IF NOT (NEW.Credits BETWEEN 0 AND 5)
  THEN SIGNAL SQLSTATE 'HY000'
      SET MYSQL_ERRNO = 1525, MESSAGE_TEXT = 'Invalid Credits; Range is [0,5]!';
  END IF;
  SET NEW.Title = TRIM(LEADING ' ' FROM NEW.Title);  #change value to be inserted
END $$
DELIMITER ;
```

Trigger Test of Course_Before_Insert

INSERT Course **VALUES** ('CS-350','Data Warehouse','Comp. Sci.',3); # Inserted OK
INSERT Course **VALUES** ('CS3700',' Temporal Databases','Comp. Sci.',5); # Error Code1525
SHOW WARNINGS;

Level	Code	Message
Error	1525	Invalid CourseID; Format is LL(L)-DDD

INSERT Course **VALUES** ('CS-3700',' Temporal Databases','Comp. Sci.',5); # Error Code1525
SHOW WARNINGS;

Level	Code	Message
Error	1525	Invalid CourseID; Format is LL(L)-DDD

INSERT Course **VALUES** ('CS-370',' Temporal Databases','Comp. Sci.',7); # Error Code1525
SHOW WARNINGS;

Level	Code	Message
Error	1525	Invalid Credits; Range is [0,5]!

INSERT Course **VALUES** ('CS-370',' Temporal Databases','Comp. Sci.',5); # Inserted OK
SELECT * FROM Course **WHERE** DeptName = 'Comp. Sci.'; # No leading '_ '

CourseID	Title	DeptName	Credits
CS-350	Data Warehouse	Comp. Sci.	3
CS-370	Temporal Databases	Comp. Sci.	5

User-defined Error Handling

- To raise a signal for a user-defined condition, use the SIGNAL statement.

- To declare a handler, use the DECLARE ... HANDLER statement.

  DECLARE *handler_action* **HANDLER**

  FOR *condition-value* *statement*

  - *Condition-value* = *mysql_or_mariadb_error_code* | **SQLSTATE** *sqlstate_value* | **SQLWARNING** | **NOT FOUND** | **SQLEXCEPTION** | …
  - If the *condition-value* occurs, the specified *statement* executes.
  - The *handler_action* indicates what action the handler takes after execution of the statement:
    - **CONTINUE**: Execution of the current program continues.
    - **EXIT**: Execution terminates for the BEGIN ... END compound statement in which the handler is declared.

- Example of a handler: see demo exercise 5.1.4

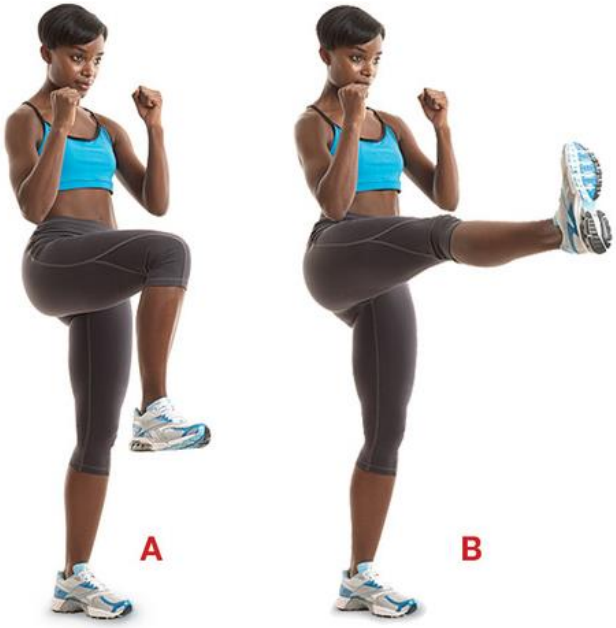
SQL Access from a Programming Language

- Application Programs
  - Often the application user interface and application logic are made in a programming language like Java, C, or Cobol.
- Application Programs can interact with a database
  - Using an Application Program Interface (API) providing functionality for
    - connecting with the database server
    - sending SQL commands to the database server
    - fetching tuples of a query result one-by-one into program variables
- Java DataBase Connectivity (JDBC)
  - API for Java
- Open DataBase Connectivity (ODBC)
  - Standard API. Works with a wide range of programming languages like C, C++, C#, Cobol and Visual Basic.

Embedded SQL

- The SQL standard defines embedding's of SQL in a variety of programming languages such as C, Java, and Cobol.
  - A language to which SQL queries are embedded is referred to as a *host language*, and the SQL structures permitted in the host language constitute *embedded SQL*.
  - Special statements are used to identify embedded SQL, like:
    - **EXEC SQL** <embedded SQL statement > **END_EXEC** for embedded Cobol
    - **# SQL** { embedded SQL statement } for embedded Java
  - A *precompiler* replaces the SQL statements in the application program with appropriate host language declarations and procedure calls that allow runtime execution of the database access.
  - This approach differs from JDBC and ODBC that use a dynamic SQL approach which does not use a precompiler, but allows the application program to construct and submit SQL queries as strings at runtime.

Demo Exercises



Demo Exercises clarify ideas and concepts from the Lecture to provide you with good Database Skills.

Discuss and do the Demo Exercises.
Solutions can be found on later slides.

Functions

5.1.1 Create a Function

Create a function which can check whether a year is a **leap year**, and then test the function.

Hint: *Year* is a leap year if

(@Year % 4 = 0) AND ((@Year % 100 <> 0) OR (@Year % 400 = 0)),
where *n%m* is the remainder of *n/m* (e.g. 13%5 = 3)

Examples:

1972 is a leap year as (True AND (True OR False)) evaluates to True;

1900 is not a leap year (True AND (False OR False)) evaluates to False;

2000 is a leap year as (True AND (False OR True)) evaluates to True;

3000 is not a leap year (True AND (False OR False)) evaluates to False;

Triggers

5.1.2 Create a Trigger

named Instructor_After_Insert that after an instructor has been inserted into the Instructor table will insert the same row in a table named InstLog with a timestamp added.

Hint, for a timestamp **NOW**(6):

SELECT NOW(6);

NOW(6)
2015-09-14 09:48:09.949155

Procedures

5.1.3 Create a Procedure

As a continuation of 5.1.2, create a procedure called InstBackup that will copy all rows from the Instructor table to a table called InstOld, and thereafter also delete all rows in the table InstLog.

The procedure should delete the rows in the old backup table (i.e. InstOld), make a new backup table with the rows of the Instructor table, and delete the rows in InstLog to prepare for recording new inserts into Instructor.

Remark: in MariaDB/MySQL, in order to be allowed to make a DELETE without a where clause, you need to disable safe mode (if not already done): **SET** SQL_SAFE_UPDATES = 0;

Procedures & Transactions

5.1.4 Create a Procedure with a Transaction

As a continuation of 5.1.3, redefine the procedure called InstBackup so that all data manipulations are done within a transaction. Test it.

```
DROP TABLE IF EXISTS InstOld;
DROP TABLE IF EXISTS InstLog;
CREATE TABLE InstOld LIKE Instructor;
CREATE TABLE InstLog LIKE Instructor;
ALTER TABLE InstLog ADD LogTime TIMESTAMP(6);

DELIMITER //
CREATE PROCEDURE InstBackup1 ()
BEGIN
 DECLARE vSQLSTATE CHAR(5) DEFAULT '00000';
 DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
   # this handler statement below ensures that
   # if an exception is raised by SQL during the transaction
   # then vSQLSTATE will be assigned a value <> '00000‘
   # and continue
   BEGIN
     GET DIAGNOSTICS CONDITION 1
     vSQLSTATE = RETURNED_SQLSTATE;
   END;
  … fill out here
END //
DELIMITER ;

… test it …
```

Events

5.1.5 Create an Event

As a continuation of 5.1.4, create an event named InstEvent that will execute the called InstBackup every week the night between Saturday and Sunday at 00:00:01, first time 2016-02-21.

Remember to set the GLOBAL Event_Scheduler to 1, to turn the event on.

Events

5.1.6 Create an Event making a dice roll

Design a Gambling Machine which rolls a dice every 5 seconds:

1. Set the GLOBAL Event_Scheduler to 1.

2. Create a table DiceRolls with attributes RollNo and DiceEyes of data type integer.

Hint: If you add AUTO_INCREMENT after the type of the RollNo attribute, then each time you insert a new row in the table, you need only stating the value of DiceEyes – the value of RollNo will automatically be generated: First time it will be 1, then 2, etc.

3. Create an event RollDice that executes every 5 seconds and inserts a random number of 1, 2, 3, 4, 5 or 6 (DiceEyes) into table DiceRolls.

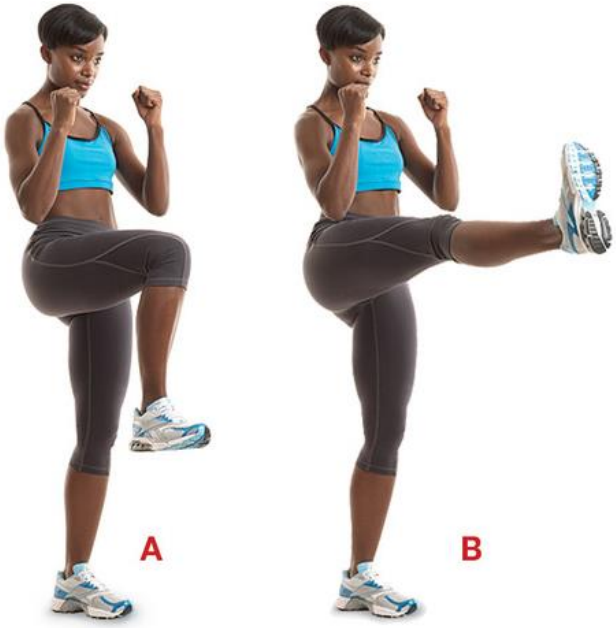
Hint:

RAND() returns a random floating-point value v in the range 0 <= v < 1.0.

FLOOR() returns the largest integer value not greater than the argument.

Make a query showing the number of times the six dice values have been played within the first 10 rolls.

Solutions to Demo Exercises



Functions

5.1.1 Create a Function

Create a function which can check whether a year is a **leap year**, and then test the function.

Hint: *Year* is a leap year if

(@Year % 4 = 0) AND ((@Year % 100 <> 0) OR (@Year % 400 = 0)),
where *n%m* is the remainder of *n/m* (e.g. 13%5 = 3)

Examples:

1972 is a leap year as (True AND (True OR False)) evaluates to True;

1900 is not a leap year (True AND (False OR False)) evaluates to False;

2000 is a leap year as (True AND (False OR True)) evaluates to True;

3000 is not a leap year (True AND (False OR False)) evaluates to False;

```
CREATE FUNCTION LeapYear ( vYear YEAR ) RETURNS BOOLEAN
RETURN
(vYear % 4 = 0) AND ((vYear % 100 <> 0) OR (vYear % 400 = 0));
```

```
SELECT LeapYear(1964);
```

Leapyear(1964)
1

```
SELECT
  StudID, StudName, Birth, Age(Birth) AS Age, LeapYear(YEAR(Birth))
AS LeapYear
FROM Student;
```

StudID	StudName	Birth	Age	LeapYear
00128	Zhang	1992-04-18	23	1
12345	Shankar	1995-12-06	20	0
19991	Brandt	1993-05-24	22	0
23121	Chavez	1992-04-18	23	1
44553	Peltier	1995-10-18	20	0
45678	Levy	1995-08-01	20	0
54321	Williams	1995-02-28	20	0

Triggers

5.1.2 Create a Trigger

named Instructor_After_Insert that after an instructor has been inserted into the Instructor table will insert the same row in a table named InstLog with a timestamp added.

Hint, for a timestamp **NOW**(6):

SELECT NOW(6);

NOW(6)
2015-09-14 09:48:09.949155

```
CREATE TABLE InstLog LIKE Instructor;
ALTER TABLE InstLog ADD LogTime  TIMESTAMP(6);

DELIMITER //
CREATE TRIGGER  Instructor_After_Insert
AFTER INSERT ON Instructor FOR EACH ROW
BEGIN
  INSERT InstLog VALUES (New.InstID,
                         New.InstName,
                         New.DeptName,
                         New.Salary,
                         NOW(6));
END //
DELIMITER ;

SELECT * FROM Instructor;
SELECT * FROM InstLog;

INSERT Instructor VALUES
    ('11001', 'Valdez', 'Comp. Sci.', 36000),
    ('11002', 'Koerver', 'Comp. Sci.', 36000);

SELECT * FROM Instructor;
SELECT * FROM InstLog;
```

InstID	InstName	DeptName	Salary	LogTime
11001	Valdez	Comp. Sci.	36000.00	2015-09-14 09:59:36.122563
11002	Koerver	Comp. Sci.	36000.00	2015-09-14 09:59:36.122563

Procedures

5.1.3 Create a Procedure

As a continuation of 5.1.2, create a procedure called InstBackup that will copy all rows from the Instructor table to a table called InstOld, and thereafter also delete all rows in the table InstLog.

The procedure should delete the rows in the old backup table (i.e. InstOld), make a new backup table with the rows of the Instructor table, and delete the rows in InstLog to prepare for recording new inserts into Instructor.

Remark: in MariaDB/MySQL, in order to be allowed to make a DELETE without a where clause, you need to disable safe mode (if not already done): **SET** SQL_SAFE_UPDATES = 0;

```
CREATE TABLE InstOld LIKE Instructor;


DELIMITER //
CREATE PROCEDURE InstBackup ()
BEGIN
  DELETE FROM InstOld; # see remark in left column
  INSERT INTO InstOld SELECT * FROM Instructor;
  DELETE FROM InstLog;
END //
DELIMITER ;

SELECT * FROM Instructor;
SELECT * FROM InstOld;
SELECT * FROM InstLog;

SET SQL_SAFE_UPDATES = 0;
CALL InstBackup;
SET SQL_SAFE_UPDATES = 1;

SELECT * FROM Instructor;
SELECT * FROM InstOld; # contains the Instructor rows
SELECT * FROM InstLog; # no rows
```

Procedures & Transactions

5.1.4 Create a Procedure with a Transaction

As a continuation of 5.1.3, redefine the procedure called InstBackup so that all data manipulations are done within a transaction. Test it.

```
DROP TABLE IF EXISTS InstOld; DROP TABLE IF EXISTS InstLog;
CREATE TABLE InstOld LIKE Instructor; CREATE TABLE InstLog LIKE Instructor;
ALTER TABLE InstLog ADD LogTime TIMESTAMP(6);

DELIMITER //
CREATE PROCEDURE InstBackup1 ()
BEGIN
 DECLARE vSQLSTATE CHAR(5) DEFAULT '00000';
 DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
   # this handler statement below ensures that
   # if an exception is raised by SQL during the transaction
   # then vSQLSTATE will be assigned a value <> '00000'
   # and continue
   BEGIN
     GET DIAGNOSTICS CONDITION 1
     vSQLSTATE = RETURNED_SQLSTATE;
   END;
 START TRANSACTION;
 DELETE FROM InstOld;
 INSERT INTO InstOld SELECT * FROM Instructor;
 DELETE FROM InstLog;
 SELECT vSQLSTATE;
 IF vSQLSTATE = '00000' THEN COMMIT;
    ELSE ROLLBACK;
 END IF;
END //
DELIMITER ;

# Test the procedure before and after "DROP TABLE InstLog;".
INSERT Instructor VALUES  ('10000', 'Hansen', 'Comp. Sci.', 50000);
SELECT * FROM InstLog; # contains the new row
SET SQL_SAFE_UPDATES = 0;
CALL InstBackup1; # SELECT vSQLSTATE returns 00000 and the transaction is committed
SELECT * FROM Instructor;
SELECT * FROM InstOld; #same as Instructor
SELECT * FROM InstLog; #no rows

DROP TABLE InstLog;
CALL InstBackup1; # SELECT vSQLSTATE returns 42S02 and the transaction is rolled back as Instlog does not exist

# Remove "SELECT vSQLSTATE;" inside the procedure when testing has been done!
```

Events

5.1.5 Create an Event

As a continuation of 5.1.4, create an event named InstEvent that will execute the called InstBackup every week the night between Saturday and Sunday at 00:00:01, first time 2016-02-21.

Remember to set the GLOBAL Event_Scheduler to 1, to turn the event on.

```
#re-create table deleted in 5.1.4 solution
CREATE TABLE InstLog LIKE Instructor;
ALTER TABLE InstLog ADD LogTime  TIMESTAMP(6);

CREATE EVENT InstEvent
ON SCHEDULE EVERY 1 WEEK
STARTS '2016-02-21 00:00:01'
DO CALL InstBackup;

SET GLOBAL event_scheduler = 1;
SHOW VARIABLES LIKE 'event_scheduler';
```

Variable_name	Value
event_scheduler	ON

Events

5.1.6 Create an Event making a dice roll

Design a Gambling Machine which rolls a dice every 5 seconds:

1. Set the GLOBAL Event_Scheduler to 1.

2. Create a table DiceRolls with attributes RollNo and DiceEyes of data type integer.

Hint: If you add AUTO_INCREMENT after the type of the RollNo attribute, then each time you insert a new row in the table, you need only stating the value of DiceEyes – the value of RollNo will automatically be generated: First time it will be 1, then 2, etc.

3. Create an event RollDice that executes every 5 seconds and inserts a random number of 1, 2, 3, 4, 5 or 6 (DiceEyes) into table DiceRolls.

Hint:

RAND() returns a random floating-point value v in the range 0 <= v < 1.0.
FLOOR() returns the largest integer value not greater than the argument.

Make a query showing the number of times the six dice values have been played within the first 10 rolls.

```
SET GLOBAL event_scheduler = 1;

CREATE TABLE DiceRolls (
RollNo INTEGER AUTO_INCREMENT PRIMARY KEY,
DiceEyes INTEGER);

CREATE EVENT RollDice
ON SCHEDULE EVERY 5 SECOND
DO
INSERT DiceRolls (DiceEyes) VALUES
(1+FLOOR(6*RAND()));

SELECT DISTINCT DiceEyes, Count(DiceEyes)
FROM DiceRolls WHERE RollNo <= 10
GROUP BY DiceEyes;

#stop event after use
SET GLOBAL event_scheduler = 0;
```

Exercises

Please answer all exercises to demonstrate your Database Skills.

Solutions are available at 11:45

Functions

5.2.1 Create a Function

1. Create a function named *BuildingCapacityFct* which takes as input a *Building* of the *Classroom* table in the *University* database and returns the total capacity of the building.
2. Test the function (i.e. execute an SQL command containing a function call of that function).

Triggers, Procedures & Error Signalling

5.2.2 Procedure with Error Signalling

For the *TimeSlot* table of the *University* database, state informally constraints (besides the key constraint) that should hold between the values of attributes in a single row and between values of attributes of two rows. Check your suggestion with a teaching assistant before continuing.

Create a procedure *InsertTimeSlot* for inserting a row into the *TimeSlot* table. It should signal an error in case the insertion of the new tuple leads to a violation of the constraints. (The procedure should not check whether the constraints already hold before the insertion.) To make the procedure more readable, it is advisable to define one or several auxiliary/helper functions to express the error conditions.

Test systematically the auxiliary functions and procedure.

5.2.3 Trigger with Error Signalling

Make a trigger TimeSlot_Before_Insert, which automatically raises a signal when inserting a row into TimeSlot (directly with an INSERT without using the InsertTimeSlot procedure), if the insertion of the new row leads to a violation of the constraints identified in the previous exercise.

Test the trigger by making INSERTs into TimeSLOT.

Events

5.2.4 Create an Event for a European Roulette

Design a Gambling Machine which rolls a ball on a European Roulette every 10 seconds and stores the Lucky number.

Create a table called BallRolls with attributes RollNo and LuckyNo.

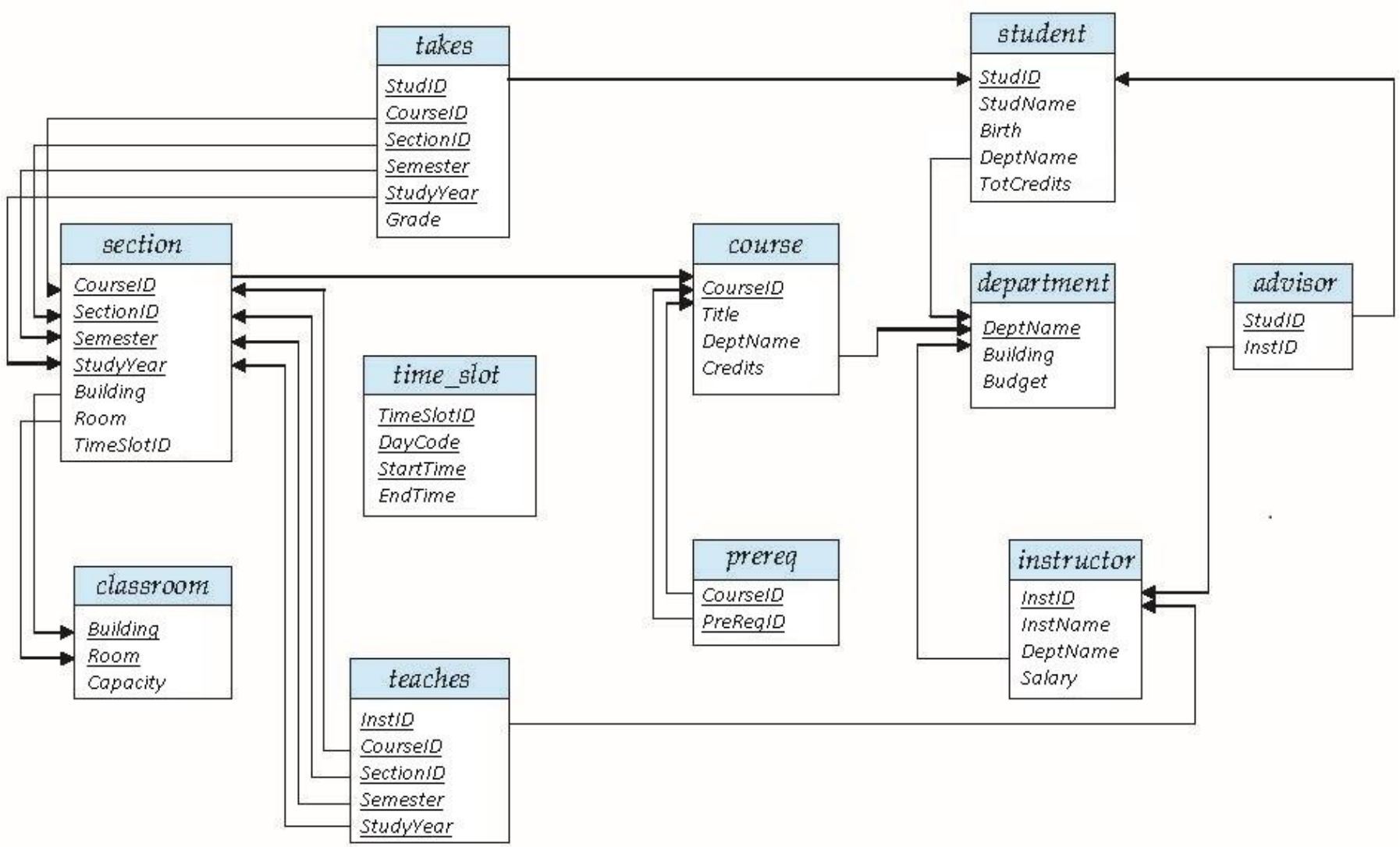
Create an event *RollBall* that executes every 10 seconds and inserts RollNo (automatically counting from 1) and LuckyNo (i.e. random number between 0 and 36) into the table BallRolls.

Appendix: University Database

- Database example used throughout the textbook and in this course!

1. Database Schema Diagram
2. Database Schema (CREATE TABLE Commands)
3. Database Instance

Database Schema Diagram from the book, but with new names



University Database Schema (1 of 4)

```
CREATE TABLE Instructor
             (InstID                    VARCHAR(5),
              InstName                  VARCHAR(20) NOT NULL,
              DeptName                  VARCHAR(20),
              Salary                    DECIMAL(8,2),
              PRIMARY KEY (InstID),
              FOREIGN KEY(DeptName) REFERENCES Department(DeptName) ON DELETE SET NULL
             );

CREATE TABLE Student
             (StudID                    VARCHAR(5),
              StudName                  VARCHAR(20) NOT NULL,
              Birth                     DATE,
              DeptName                  VARCHAR(20),
             TotCredits                 DECIMAL(3,0),
              PRIMARY KEY(StudID),
              FOREIGN KEY(DeptName) REFERENCES Department(DeptName) ON DELETE SET NULL
             );

CREATE TABLE Advisor
             (StudID                    VARCHAR(5),
              InstID                    VARCHAR(5),
              PRIMARY KEY(StudID),
              FOREIGN KEY(InstID) REFERENCES Instructor(InstID) ON DELETE SET NULL,
              FOREIGN KEY(StudID) REFERENCES Student(StudID) ON DELETE CASCADE
             );

CREATE TABLE Department
             (DeptName                  VARCHAR(20),
              Building                  VARCHAR(15),
              Budget                    DECIMAL(12,2),
              PRIMARY KEY(DeptName)
             );
```

University Database Schema (2 of 4)

```
CREATE TABLE Course
             (CourseID                  VARCHAR(8),
              Title                     VARCHAR(50),
              DeptName                  VARCHAR(20),
              Credits                   DECIMAL(2,0),
              PRIMARY KEY(CourseID),
              FOREIGN KEY(DeptName) REFERENCES Department(DeptName) ON DELETE SET NULL
             );


CREATE TABLE PreReq
             (CourseID                  VARCHAR(8),
              PreReqID                  VARCHAR(8),
              PRIMARY KEY(CourseID, PreReqID),
              FOREIGN KEY(CourseID) REFERENCES Course(CourseID) ON DELETE CASCADE,
              FOREIGN KEY(PreReqID) REFERENCES Course(CourseID)
);
```

University Database Schema (3 of 4)

```
CREATE TABLE Teaches
             (InstID                    VARCHAR(5),
              CourseID                  VARCHAR(8),
              SectionID                 VARCHAR(8),
              Semester                  ENUM('Fall','Winter','Spring','Summer'),
              StudyYear                 YEAR,
              PRIMARY KEY(InstID, CourseID, SectionID, Semester, StudyYear),
              FOREIGN KEY(CourseID, SectionID, Semester, StudyYear)
                  REFERENCES Section(CourseID, SectionID, Semester, StudyYear) ON DELETE CASCADE,
              FOREIGN KEY(InstID) REFERENCES Instructor(InstID) ON DELETE CASCADE
             );

CREATE TABLE Takes
             (StudID                    VARCHAR(5),
              CourseID                  VARCHAR(8),
              SectionID                 VARCHAR(8),
              Semester                  ENUM('Fall','Winter','Spring','Summer'),
              StudyYear                 YEAR,
              Grade                     VARCHAR(2),
              PRIMARY KEY(StudID, CourseID, SectionID, Semester, StudyYear),
              FOREIGN KEY(CourseID, SectionID, Semester, StudyYear)
                  REFERENCES Section(CourseID, SectionID, Semester, StudyYear) ON DELETE CASCADE,
              FOREIGN KEY(StudID) REFERENCES Student(StudID) ON DELETE CASCADE
             );
```

University Database Schema (4 of 4)

```
CREATE TABLE Section
             (CourseID                  VARCHAR(8),
              SectionID                 VARCHAR(8),
              Semester                  ENUM('Fall','Winter','Spring','Summer'),
              StudyYear                 YEAR,
              Building                  VARCHAR(15),
              Room                      VARCHAR(7),
              TimeSlotID                VARCHAR(4),
              PRIMARY KEY(CourseID, SectionID, Semester, StudyYear),
              FOREIGN KEY(CourseID) REFERENCES Course(CourseID) ON DELETE CASCADE,
              FOREIGN KEY(Building, Room) REFERENCES Classroom(Building, Room) ON DELETE SET NULL
             );

CREATE TABLE TimeSlot
             (TimeSlotID  VARCHAR(4),
              DayCode                   ENUM('M','T','W','R','F','S','U'),
              StartTime                 TIME,
              EndTime                   TIME,
              PRIMARY KEY(TimeSlotID, DayCode, StartTime)
             );

CREATE TABLE Classroom
             (Building                  VARCHAR(15),
              Room                      VARCHAR(7),
              Capacity                  DECIMAL(4,0),
              PRIMARY KEY(Building, Room)
             );
```

Database Instance (1 of 4)

Instructor

InstID	InstName	DeptName	Salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Student

StudID	StudName	Birth	DeptName	TotCredits
00128	Zhang	1992-04-18	Comp. Sci.	102
12345	Shankar	1995-12-06	Comp. Sci.	32
19991	Brandt	1993-05-24	History	80
23121	Chavez	1992-04-18	Finance	110
44553	Peltier	1995-10-18	Physics	56
45678	Levy	1995-08-01	Physics	46
54321	Williams	1995-02-28	Comp. Sci.	54
55739	Sanchez	1995-06-04	Music	38
70557	Snow	1995-11-22	Physics	0
76543	Brown	1994-03-05	Comp. Sci.	58
76653	Aoi	1993-09-18	Elec. Eng.	60
98765	Bourikas	1992-09-23	Elec. Eng.	98
98988	Tanaka	1992-06-02	Biology	120

Advisor

StudID	InstID
12345	10101
44553	22222
45678	22222
00128	45565
76543	45565
23121	76543
98988	76766
76653	98345
98765	98345

Department

DeptName	Building	Budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

Database Instance (2 of 4)

Teaches

InstID	CourseID	SectionID	Semester	StudyYear
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
10101	CS-101	1	Fall	2009
45565	CS-101	1	Spring	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
10101	CS-315	1	Spring	2010
45565	CS-319	1	Spring	2010
83821	CS-319	2	Spring	2010
10101	CS-347	1	Fall	2009
98345	EE-181	1	Spring	2009
12121	FIN-201	1	Spring	2010
32343	HIS-351	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

Takes

StudID	CourseID	SectionID	Semester	StudyYear	Grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	NULL

Database Instance (3 of 4)

Course

CourseID	Title	DeptName	Credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

PreReq

CourseID	PreReqID
BIO-301	BIO-101
BIO-399	BIO-101
CS-190	CS-101
CS-315	CS-101
CS-319	CS-101
CS-347	CS-101
EE-181	PHY-101

Database Instance (4 of 4)

Section

CourseID	SectionID	Semester	StudyYear	Building	Room	TimeSlotID
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Classroom

Building	Room	Capacity
Packard	101	500
Painter	514	10
Taylor	3128	70
Watson	100	30
Watson	120	50

TimeSlot

TimeSlotID	DayCode	StartTime	EndTime
A	M	08:00:00	08:50:00
A	W	08:00:00	08:50:00
A	F	08:00:00	08:50:00
B	M	09:00:00	09:50:00
B	W	09:00:00	09:50:00
B	F	09:00:00	09:50:00
C	M	11:00:00	11:50:00
C	W	11:00:00	11:50:00
C	F	11:00:00	11:50:00
D	M	13:00:00	13:50:00
D	W	13:00:00	13:50:00
D	F	13:00:00	13:50:00
E	T	10:30:00	11:45:00
E	R	10:30:00	11:45:00
F	T	14:30:00	15:45:00
F	R	14:30:00	15:45:00
G	M	16:00:00	16:50:00
G	W	16:00:00	16:50:00
G	F	16:00:00	16:50:00
H	W	10:00:00	12:30:00